

ScriptLens — Architecture Specification v3.0

(Structure-Only)

Version	3.0 — Structure-only product
Date	July 2026
Status	Final specification — pre-implementation
Audience	Solo builder, future contributors, product review
Supersedes	ARCHITECTURE.md v2.4 (contradiction + extension-first)

Table of contents

1. [Executive summary](#)
2. [Product definition](#)
3. [Scope lock](#)
4. [System architecture](#)
5. [Thin client \(browser\)](#)
6. [Backend \(FastAPI\)](#)
7. [Core engine — scene dependency](#)
8. [Simulate cut](#)
9. [Simulate edit](#)
10. [Orphan scenes](#)
11. [PDF ingestion](#)
12. [API contracts](#)
13. [Data models](#)
14. [User workflows](#)
15. [UI architecture](#)
16. [Security and privacy](#)
17. [Performance and reliability](#)
18. [Deployment — Hostinger](#)
19. [Implementation status](#)
20. [Build sequence \(solo\)](#)
21. [Out of scope](#)
22. [Appendix A — System diagram](#)
23. [Appendix B — Competitive positioning](#)
24. [Appendix C — Glossary](#)

1. Executive summary

ScriptLens is a **structural editing intelligence layer** for screenplays. Writers upload a script and receive:

1. **Orphan scene count** — scenes that sit loosely in the story graph
2. **Simulate cut** — preview what downstream scenes break if one scene is removed
3. **Simulate edit** — preview how dependency edges change when scene text is rewritten

All analysis runs on the server. The customer browser is a thin client. Writers on **4 GB laptops** with many tabs open are supported because spaCy, PDF parsing, and graph logic never run on the client.

Plot contradiction detection is removed from the product. The `plot_contradiction.py` module remains in the repository for internal regression tests but is **not called** by the v3 product path.

Product promise (one sentence)

Upload your script, see loose scenes, and preview what breaks if you cut or rewrite — without changing your original file.

Architecture principles

Thin client	Browser sends uploads and simulation requests; server returns JSON
Non-destructive	Original file never modified; all operations are preview
Deterministic core	Scene graph and impact paths are explainable, not LLM-black-box
No LLM in v1	Zero API cost; high margin; fast responses
Solo-deployable	Hostinger VPS + static HTML + single FastAPI process

2. Product definition

2.1 Target user

Working screenwriters	“Can I cut this scene?”
Writing partners	“What happens if we change this beat?”
Development readers	Quick structural map before notes

2.2 Core value proposition

“If I cut this scene, what breaks?”	Simulate cut + dependency paths
“If I rewrite this scene, what connections change?”	Simulate edit + edge diff
“Which scenes sit loosely in the story?”	Orphan count + scroll-to-scene
“Where is that scene?”	Scene list + Go to scene from impact panel

2.3 What ScriptLens is not

- Not a screenplay editor (writers stay in Final Draft / WriterDuet)
- Not a continuity police tool (no “you made a mistake” flags in v3)
- Not a production breakdown tool (no cast/props scheduling)
- Not a story-universe platform (no franchise canon DB)
- Not a local Chrome extension with on-device NLP (v3 is web-first)

2.4 Platform (v3)

Frontend	Static HTML + CSS + vanilla JavaScript on Hostinger
Backend	FastAPI + uvicorn on Hostinger VPS
Engine	Existing <code>scriptlensCore</code> Python modules
CLI	<code>run_scriptlens.py --structure-only</code> for dev smoke tests

3. Scope lock

3.1 In scope (v3.0)

Upload Fountain / PDF	Yes
Scene list + script reader	Yes
Orphan count + list	Yes
Simulate cut	Yes
Simulate edit	Yes
Go-to-scene from impact panel	Yes
PDF auto <code>refined</code> conversion	Yes
Full / Limited structure mode banner	Yes
Server-side session (in-memory)	Yes

3.2 Out of scope (v3.0 — do not build)

Plot contradiction UI or API	Removed
Health score from contradiction count	Removed
Cast / props panels (especially on PDF)	Hidden
Chrome extension	v3.1+

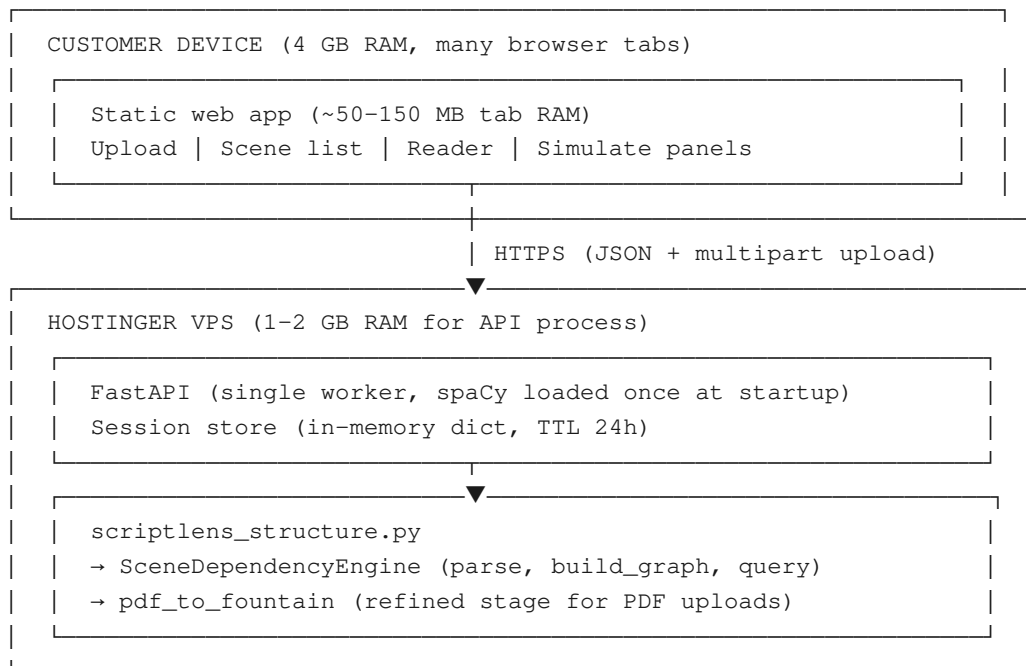
User accounts / Supabase	v3.1+
Stripe billing	v3.1+
Redis / Postgres	v3.1+
Tier 3 LLM summaries	Never in structure-only path
Final Draft <code>.fdx</code> native import	Export to PDF/Fountain first

3.3 Repository modules — product vs internal

<code>scene_dependency.py</code>	Yes	—
<code>scriptlens_structure.py</code>	Yes (new)	—
<code>pdf_to_fountain.py</code>	Yes	—
<code>pdf_screenplay_loader.py</code>	Yes	—
<code>nlp_shared.py</code>	Yes	—
<code>plot_contradiction.py</code>	No	Corpus regression
<code>scriptlens_analyser.py</code> (full)	No	Legacy CLI

4. System architecture

4.1 High-level topology



4.2 Zone responsibilities

Browser	Upload UI, scene selection, reader render, simulate UX	NLP, PDF extraction, graph math
API server	Parse, graph build, simulate cut/edit, session	Long-term user accounts (v3)
Engine	Deterministic dependency edges	LLM calls, contradiction rules

4.3 Request lifecycle

POST /upload

- save text in session[script_id]
- parse_fountain_text
- build_graph (no ContradictionEngine)
- return { script_id, scenes[], orphan_count, structure_mode }

POST /simulate/cut { scene_id }

- get_delete_impact(scene_id)
- return impacted scenes + paths

POST /simulate/edit { scene_id, modified_text }

- splice text → re-parse → rebuild graph
- diff edges before/after
- return edge_diff + impact_delta

5. Thin client (browser)

5.1 Design constraints for 4 GB customer laptops

No client-side NLP	All analysis server-side
No client-side PDF parsing	File sent as multipart upload
Small JS bundle	Vanilla JS; no React for v3
Lazy scene rendering	Render visible scenes only (virtual scroll)
No large in-memory script copy	Server holds canonical text; client holds scene summaries

5.2 Estimated client RAM

Static page + JS	~20–40 MB
Scene list (210 items)	~5 MB
Visible script blocks (lazy)	~30–80 MB
Total tab	~50–150 MB

5.3 Frontend file structure (planned)

```
web/
├─ index.html      # Upload + workspace shell
├─ app.js          # API client, state, simulate handlers
├─ reader.js       # Lazy scene renderer
└─ styles.css      # 3-panel layout (280px | flex | 320px)
```

5.4 CORS

API allows the Hostinger frontend origin only. No credentials cookies in v3.

6. Backend (FastAPI)

6.1 Process model

Workers	1	spaCy singleton; 8 GB dev / 1–2 GB VPS
spaCy load	At startup	get_shared_nlp() once per process
Model	en_core_web_sm	Smallest viable; ~100 MB Python heap
Session store	dict[str, AnalysisSession]	No Redis in v3
Session TTL	24 hours	Privacy + memory bound
Max upload size	15 MB	Typical screenplay PDF

6.2 AnalysisSession (server-side)

```
@dataclass
class AnalysisSession:
    script_id: str          # SHA256 prefix of normalized text
    screenplay_text: str     # Canonical fountain text
    scenes: list[SceneBlock]
    engine: SceneDependencyEngine
    structure_mode: str     # "full" | "limited"
    created_at: datetime
```

6.3 New module: scriptlens_structure.py

Thin wrapper — **no import of ContradictionEngine**.

```
def analyze_structure(screenplay_text: str) -> StructureReport:
    nlp = get_shared_nlp()
    engine = SceneDependencyEngine(nlp=nlp)
    scenes = engine.parse_fountain_text(screenplay_text)
    engine.build_graph(scenes) # fact_store=None
    return StructureReport(
        scenes=scenes,
        orphans=engine.get_orphan_scenes(),
        graph_summary=engine.export_graph_summary(),
        high_risk_scenes=rank_by_delete_impact(engine, scenes),
```

)

6.4 API directory (planned)

```
api/
├─ main.py           # FastAPI app, CORS, startup spaCy load
├─ routes/
│   ├─ upload.py
│   ├─ orphans.py
│   └─ simulate.py
├─ sessions.py      # In-memory store + TTL sweep
└─ schemas.py       # Pydantic request/response models
```

7. Core engine — scene dependency

7.1 Purpose

Build a **directed scene graph** where edges represent narrative dependencies (characters, objects, locations, introductions) between earlier and later scenes.

7.2 Existing implementation

Fountain parser	scene_dependency.py	Done
Graph builder	scene_dependency.py	Done
Orphan query	get_orphan_scenes()	Done
Delete impact	get_delete_impact()	Done
Graph summary	export_graph_summary()	Done
Shared spaCy	nlp_shared.py	Done

7.3 Graph model

Node	One per scene (scene_id, e.g. scene_047)
Edge	Directed from_scene → to_scene with weight, edge_type, explanation
Order	Edges only from earlier scene to later scene (no cycles)
Merge	Multiple signals between same pair → weights summed

7.4 Edge types (v3)

Character presence	0.5–0.7	Shared character across scenes
--------------------	---------	--------------------------------

Object introduction	0.8–1.0	Object introduced, reused later
Location continuity	0.4	Location from heading repeats
Character fact	0.6	Established status/trait in action

7.5 Query operations (product-facing)

Orphans	<code>get_orphan_scenes()</code>	Left panel count
Simulate cut	<code>get_delete_impact(scene_id)</code>	Right panel impact list
Upstream deps	<code>get_scene_dependencies(scene_id)</code>	v3.1 optional
High-risk rank	Derived from impact count	Optional “do not cut lightly” hint

8. Simulate cut

8.1 Definition

Read-only preview of removing one scene. Does not modify the script.

8.2 Algorithm (implemented)

Input: graph *G*, scene_id *S*

Process:

1. `descendants = nx.descendants(G, S)`
2. For each *D* in `descendants`:
`path = shortest_path(G, S, D)`
`weight = sum(edge weights on path)`
3. Sort by weight descending

Output: list of { scene_id, heading, dependency_path, total_weight }

8.3 API response shape

```
{
  "removed_scene": {
    "scene_id": "scene_047",
    "scene_number": 47,
    "heading": "INT. KITCHEN - DAY"
  },
  "impacted_scenes": [
    {
      "scene_id": "scene_062",
      "scene_number": 62,
      "heading": "INT. CAR - NIGHT",
      "dependency_path": ["scene_047", "scene_051", "scene_062"],
      "total_weight": 1.4,
      "explanation": "object: knife"
    }
  ]
}
```


8.4 UI behavior

Before cut	Normal reader	Empty or placeholder
After cut	Removed scene = ghost overlay + “Simulated removal” chip	Impact list + Go to scene
Clear	Restore normal	Clear panel

9. Simulate edit

9.1 Definition

Read-only preview of rewriting one scene’s Fountain text. Shows how dependency edges change.

9.2 Algorithm (v3 — full rebuild)

Input: session, scene_id S, modified_scene_text T

Process:

1. edges_before = snapshot(engine.graph)
2. screenplay' = splice(scene S body → T) into full text
3. scenes' = parse_fountain_text(screenplay')
4. engine'.build_graph(scenes')
5. edges_after = snapshot(engine'.graph)
6. diff = compare(edges_before, edges_after)
7. impact_delta = scenes that lost incoming edges from S

Output: { added_edges, removed_edges, changed_weights, orphan_delta }

v3.1 optimization: Incremental rebuild for scene S only (same diff output, faster).

9.3 v3 UI scope

- Textarea pre-filled with selected scene’s raw Fountain block
- Button: **Simulate edit**
- Right panel: edge diff list (human-readable)
- No inline WYSIWYG screenplay editor in v3

9.4 API response shape

```
{
  "scene_id": "scene_047",
  "edge_diff": {
    "added": [
      { "from": "scene_047", "to": "scene_051", "type": "object", "explanation": "knife"
    },
    "removed": [
      { "from": "scene_047", "to": "scene_062", "type": "object", "explanation": "knife"
    },
    "changed": []
  ],
  "orphan_delta": { "before": 3, "after": 4 },
  "downstream_at_risk": ["scene_062", "scene_088"]
}
```

10. Orphan scenes

10.1 Definition

A scene that **introduces** narrative elements (characters, objects, locations) that **no later scene depends on** through the graph — and has **no meaningful downstream edges** from its introductions.

10.2 UX framing

"3 loosely connected scenes"	"3 errors"
"May be safe to trim — verify dramatically"	"Delete these scenes"

Montage, comedy tags, and breathers may appear as orphans — this is expected.

10.3 Interaction

- Left panel: **ORPHANS 3** (tap → filter scene list or scroll to first)
- Scene row badge: optional dot for orphan scenes

11. PDF ingestion

11.1 Pipeline

PDF upload

- `convert_pdf_to_fountain(stage="refined")`
- `analyze_structure(text)`
- `structure_mode` detection

11.2 Structure modes

Full	≥ 1 extractable INT/EXT slugline	"Full structure mode"
Limited	0 sluglines (image PDF)	"Limited structure mode — scene breaks not detected. Upload a text-based PDF or Fountain for full analysis."

11.3 Hollywood benchmark reference

Tested scripts (refined pipeline): American Beauty (162 scenes), Carrie, Almost Famous, Conclave, Batman Begins, Citizen Kane.

11.4 Product rules for PDF

Auto <code>refined</code> on upload	Always
Show cast/props list	Never in v3 UI
Contradiction pass	Never

scene_id	str	Stable ID (<code>scene_NNN</code>)
scene_number	int	Display order
heading	str	Slugline
characters	list[str]	Dialogue cues
objects	list[str]	Extracted props
locations	list[str]	From heading
action_text	str	Raw action lines
dialogue_blocks	list	Character + lines

13.2 DependencyEdge

from_scene_id	str
to_scene_id	str
weight	float
edge_type	str
explanation	str

13.3 Pydantic schemas (API layer)

- UploadResponse
- SceneSummary
- OrphanListResponse
- SimulateCutRequest / SimulateCutResponse
- SimulateEditRequest / SimulateEditResponse

14. User workflows

14.1 Primary flow

1. Land on upload page
2. Drop PDF or Fountain
3. Wait for "Building scene graph..." (10–20s for large PDF)
4. Workspace loads:
 - Orphan count (left)
 - Scene list (left)
 - Script reader (center)
5. Select scene → Simulate cut OR Edit scene
6. Review impact (right) → Go to scene
7. Clear simulation → back to normal

14.2 Non-destructive rule

The uploaded file on the writer’s machine is **never modified**. ScriptLens does not integrate with Final Draft for write-back in v3.

14.3 Session end

- User closes tab → session remains until TTL
- No account required in v3
- Privacy copy: “Your script is processed on our server and deleted within 24 hours.”

15. UI architecture

15.1 Layout (desktop, min width 1024px)

Left	280px fixed	Orphans, scene list, Simulate cut, Edit scene
Center	flex 1	Script reader (lazy render)
Right	320px fixed	Simulation results

15.2 Left panel (v3)

ORPHANS	3
SCENES	
1 INT. HOUSE - DAY	
5 EXT. ROAD - NIGHT	◀
[Simulate cut]	
[Edit scene]	

Removed from v2 UX: Plot issues count, contradictions tab, View all continuity.

15.3 Right panel — simulate cut

Impact of removing Scene 5

⚠ Scene 12 — downstream dependent
Path: 5 → 9 → 12
Object: knife
[Go to scene 12]

[Clear simulation]

15.4 Right panel — simulate edit

Changes if you edit Scene 5

- Removed link: knife → Scene 12
+ Added link: flashlight → Scene 12

Orphans: 3 → 4

[Clear edit preview]

15.5 Mobile (v3)

Read-only scene list + upload. Simulate on desktop only; banner: “Open on desktop for simulate.”

16. Security and privacy

Script sensitivity	HTTPS only; 24h TTL; no full-body logging
Logs	script_hash, scene_count, duration_ms — never screenplay text
Upload validation	Extension whitelist: .fountain, .txt, .pdf
Session isolation	script_id = hash; no cross-session access
CORS	Frontend origin only

17. Performance and reliability

17.1 Latency targets (server)

Fountain parse (120 scenes)	< 100ms	< 200ms
Graph build (120 scenes)	< 200ms	< 400ms
PDF convert + analyse (160 scenes)	< 8s	< 20s
Simulate cut	< 50ms	< 100ms
Simulate edit (full rebuild)	< 500ms	< 2s

17.2 RAM targets

Customer browser tab	50–150 MB	Thin client
API process (steady)	300–600 MB	spaCy sm + one session
API peak (210 scenes)	< 800 MB	American Pie-scale
Developer machine	8 GB OK	Close tabs during PDF dev
Hostinger VPS	≥ 1 GB (2 GB preferred)	Single worker

17.3 Failure modes

0 scenes parsed	"Could not detect scenes — try Fountain or a text-based PDF"
Limited structure mode	Banner + orphans/cut may be unreliable
Session expired	"Session expired — please re-upload"
Server timeout	Retry button; spinner max 30s

17.4 Concurrency (v3)

Single worker = one heavy analysis at a time. Optional: simple queue with "Analysing..." if second upload arrives.

18. Deployment — Hostinger

18.1 Topology

Static frontend	Hostinger <code>public_html/</code>
FastAPI	Same VPS, subdomain <code>api.yourdomain.com</code>
SSL	Hostinger Let's Encrypt
Process manager	<code>systemd</code> or <code>supervisord</code> → <code>uvicorn api.main:app --host 0.0.0.0 --port 8000 --workers 1</code>

18.2 Environment

```
PYTHONPATH=/path/to/scriptlensCore
SPACY_MODEL=en_core_web_sm
SESSION_TTL_HOURS=24
MAX_UPLOAD_MB=15
CORS_ORIGIN=https://yourdomain.com
```

18.3 Deploy checklist

- [] `pip install -r requirements.txt + python -m spacy download en_core_web_sm`
- [] `GET /api/health` returns 200
- [] Upload American Beauty PDF → `scene_count > 100`
- [] Simulate cut returns ≥ 1 impact on known demo scene
- [] CORS from production domain
- [] Privacy page linked from upload screen

18.4 Cost (year 1)

Hostinger (existing)	\$0 incremental
----------------------	-----------------

LLM APIs	\$0
Total	\$0 cash (solo build)

19. Implementation status

19.1 Done (repository today)

SceneDependencyEngine	Done
get_orphan_scenes()	Done
get_delete_impact()	Done
PDF → Fountain pipeline	Done
UX mockups (cut flow)	Done
scripts/md_to_pdf.py	Done

19.2 Not done (v3 build)

scriptlens_structure.py	Not started
api/ FastAPI service	Not started
web/ frontend	Not started
Simulate edit	Not started
Hostinger production deploy	Not started
tests/test_simulate_edit.py	Not started

20. Build sequence (solo)

1	scriptlens_structure.py + CLI --structure-only	No contradictions in output
2	FastAPI upload + orphans + health	curl upload works
3	Simulate cut endpoint	Matches expected_simulate_delete on corpus
4	Frontend upload + scene list + reader	Browser upload works

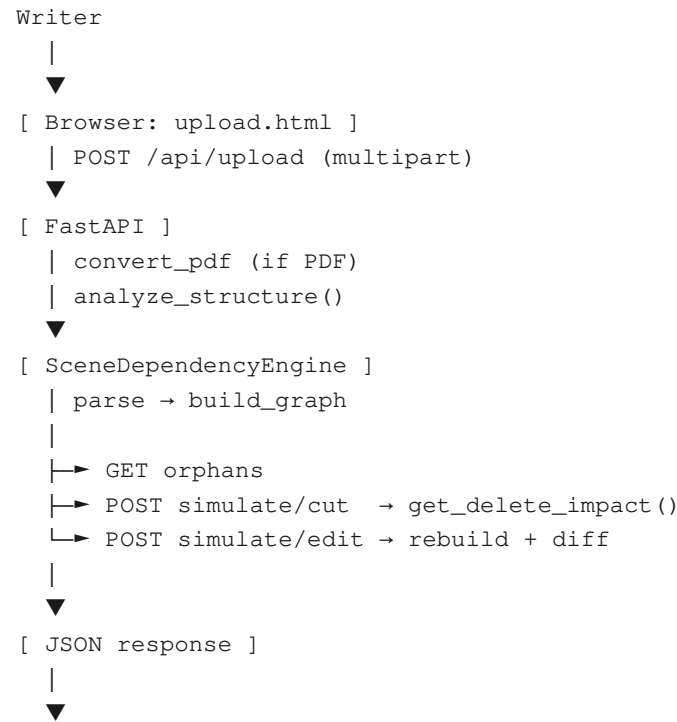
5	Simulate cut UI	Full cut flow end-to-end
6–7	Simulate edit backend + UI	Edge diff on planted edit test
8	PDF path + structure mode banner	American Beauty PDF works
9	Hostinger deploy	Public URL live
10	Demo polish	2-min screen recording

MVP milestone: Week 5 (orphans + simulate cut on web).

21. Out of scope

Plot contradiction product surface	Engine kept for CI only
Chrome extension	v3.1
Auth / billing	v3.1
.fdx import	Export to PDF first
AI rewrite / dialogue generation	Never
Production breakdown	Studiovity lane
Franchise / series canon	Othelia lane

Appendix A — System diagram



Appendix B — Competitive positioning

ScriptLens v3			
Scope	Franchise / multi-format	Writing + production	Single screenplay
Simulate cut	Beta / vague	No	Core feature
Simulate edit	Beta / vague	No	Core feature
Orphans	No	No	Core feature
Contradiction flags	Yes	No	No (removed)
Customer RAM	Server-side assumed	Local app	4 GB OK (thin client)
Price	Enterprise beta	\$250+	\$0–15/mo planned

Positioning line: “See what breaks before you cut.”

Appendix C — Glossary

Orphan scene	Scene with weak or no downstream dependency edges
Simulate cut	Preview of removing a scene without editing the file
Simulate edit	Preview of rewriting a scene’s text and its graph impact
Dependency path	Ordered list of scene IDs linking setup to payoff
Structure mode	Full (sluglines detected) vs Limited (image PDF)
Thin client	Browser UI only; no local NLP
Fountain	Plain-text screenplay format
refined	PDF conversion stage with cleanup + manual-pass rules